SMART SYMPTOM CHECKER

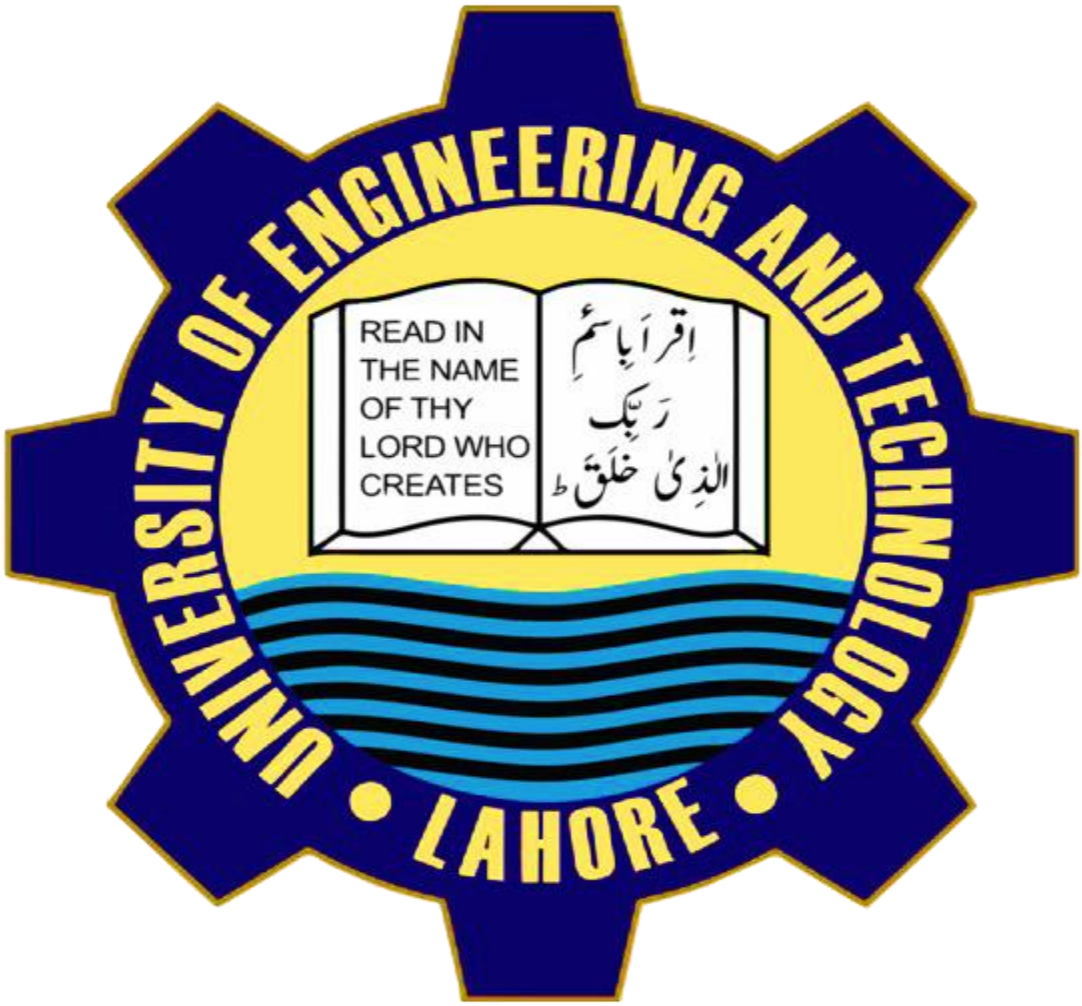
Medical Diagnosis Desktop Application

Final Year Project Report

Python | PyQt5 | SQLite | SHA-256 Security

Submitted by:
Muhammad Bilal
2026(S)-CYS-86

Submitted to:
Sir. Hafiz Mubashar Imran
Department Of Computer Engineering



Abstract

Smart Symptom Checker is an offline desktop application that helps users identify illnesses based on symptoms and provides medication recommendations and treatment advice instantly.

Smart Symptom Checker is a desktop medical assistant built with Python and PyQt5. Users select their symptoms from a checklist, and the app immediately returns a diagnosis with a confidence score, disease description, medications, home care tips, and a list of available doctors — all without any internet connection.

The application also features a separate Doctor Login portal where medical staff can view all patient records, add new symptoms to the checker, and manage the doctor directory. Data is stored securely in a local SQLite database with SHA-256 encrypted passwords.

1. Introduction

1.1 Problem Statement

Patients often struggle to identify what illness they have before seeing a doctor. Searching symptoms online gives unreliable results. There is a need for a simple, offline tool that gives instant, structured medical guidance.

1.2 Objectives

- Build a symptom-selection checklist and instant diagnosis engine.
- Implement secure patient and doctor login using SHA-256 password hashing.
- Show disease name, confidence %, medications, and home care for every diagnosis.
- Allow doctors to view patient records, manage symptoms, and update doctor listings.
- Store all data locally using SQLite — no internet required.

1.3 Scope

Single-machine Windows desktop application. Covers 6 common conditions across 15 symptoms. All data is stored locally. For educational and preliminary screening use only — not a replacement for professional medical advice.

2. System Design

2.1 Application Structure

The application follows a simple three-layer design:

Layer	What It Does
Data Layer	SQLite database stores users, patient records, doctors, and custom symptoms.
Logic Layer	The diagnose() function matches selected symptoms to disease patterns.
Presentation Layer	PyQt5 windows built from .ui files handle all screen display and user interaction.

2.2 Database Tables

Table	Stores
users	Patient accounts: name, age, gender, phone, username, hashed password.
records	Diagnosis history: patient name, condition, symptoms chosen, date/time.
doctors_auth	Doctor login credentials (username + hashed password).
available_doctors	Doctor directory: name, specialization, contact, availability.
custom_symptoms	Extra symptoms added by doctors: key and display label.

2.3 Disease Knowledge Base

DISEASE_DATA is a Python dictionary containing complete information for each condition. Every entry has: disease name, confidence score, description, transmission, recovery duration, common symptoms, doctor warning, medication list, and home care tips.

Disease	Confidence
Flu / COVID-like Illness	90%
Throat Infection (Pharyngitis)	85%
Gastroenteritis (Stomach Flu)	82%
Viral Infection	80%
Stress / Dehydration	75%
Anxiety / Panic Disorder	72%
Symptoms Unclear (fallback)	40%

3. Diagnosis Algorithm

3.1 How diagnose() Works

The diagnose() function takes a dictionary of 15 symptoms (True/False) and checks them against rules in priority order. The first rule that matches returns the disease. If nothing matches, it returns unknown with 40% confidence.

Priority	Rule (Symptom Combination)	Returns
1st	Fever AND Cough AND (Fatigue OR Body Pain)	flu_covid
2nd	(Nausea OR Vomiting) AND (Diarrhea OR Abdominal Pain)	gastro
3rd	Sore Throat AND (Cough OR Fever)	throat_infection
4th	Fever AND (Headache OR Body Pain OR Dizziness)	viral_infection
5th	Anxiety AND (Chest Pain OR Dizziness OR Shortness of Breath)	anxiety_disorder
6th	Fatigue AND Headache OR Dizziness AND Headache (no fever)	stress_dehydration
Fallback	No rule matched	unknown (40%)

4. Application Windows

Window	Purpose
LoginWindow	Patient login screen. Enter username and password to access the symptom checker.
RegisterWindow	New patient registration. Captures name, age, gender, phone, username, password.
DoctorLoginWindow	Separate login for medical staff. Opens the Doctor Dashboard on success.
SymptomsWindow	Main patient screen. 15 symptom checkboxes, search bar, live count, Predict button.
ResultWindow	Shows diagnosis: disease name, confidence bar, description, transmission, duration, warning.
TreatmentWindow	Shows medications with dosages, home care tips, and available doctors for consultation.
DoctorDashboard	Admin panel. Searchable table of all patient records, today count, total count.
ManageSymptomsWindow	Add or delete custom symptoms. Built-in symptoms are protected from deletion.
ManageDoctorsWindow	Add, edit, or delete doctors in the consultation directory.

5. Libraries and Key Concepts

5.1 Libraries Used

Library	What It Does in This Project
sys	Starts and cleanly exits the PyQt5 application event loop.
os	Builds portable file paths so the app finds its .ui and .db files anywhere.
sqlite3	Built-in Python driver for creating, reading, and writing the local database.
hashlib	Hashes passwords with SHA-256 so plain-text passwords are never stored.
datetime	Records the exact date and time for every patient diagnosis saved.
PyQt5.QtWidgets	All GUI elements: windows, buttons, checkboxes, tables, message boxes.
PyQt5.uic.loadUi	Loads pre-designed .ui layout files into Python window classes.

5.2 Object-Oriented Programming (OOP)

Every screen is a Python class that inherits from QMainWindow. Inheritance means each window automatically gets Qt window features (title bar, close button, etc.) without any extra code. Each class then adds its own buttons, logic, and data through its own methods.

- class LoginWindow(QMainWindow): — inherits all window behavior from Qt.
- self.user stores the logged-in user data and is passed between windows.
- Each window creates the next window and shows it: self.result_win = ResultWindow(...); self.result_win.show()

5.3 Event-Driven Programming

The app does not run line by line — it waits for user actions. Buttons, checkboxes, and text fields are connected to functions that run only when the user interacts with them.

- self.predictButton.clicked.connect(self.handle_predict) — runs handle_predict when Predict is clicked.
- self.searchBar.textChanged.connect(self.filter_symptoms) — filters symptoms on every keystroke.
- checkbox.stateChanged.connect(self.update_count) — updates the symptom count on every tick/untick.

6. Security and Application Workflow

6.1 Password Security with SHA-256

All passwords are hashed using SHA-256 before being stored in the database. SHA-256 is a one-way function — the hash cannot be reversed to reveal the original password. Even if someone steals the database file, they cannot read any password.

hash_password(mypass123) → stores: a94f2b3d7e1c... (64 hex characters) | Original password is never saved.

6.2 Step-by-Step Application Workflow

Step	What Happens
1	User opens the app → main() initializes the database and shows LoginWindow.
2	Patient enters username and password → app hashes the password and checks the database.
3	On success, SymptomsWindow opens. Patient sees 15 symptom checkboxes.
4	Patient ticks symptoms and clicks Predict → diagnose() runs and finds the best match.
5	ResultWindow shows: disease name, confidence %, description, transmission, and warning.
6	Patient clicks Treatment → TreatmentWindow shows medications, home care, and doctors.
7	Patient clicks Save Record → diagnosis saved to SQLite with name, condition, and timestamp.
8	Doctor logs in via Doctor Login → DoctorDashboard shows all saved patient records.
9	Doctor can search records, add/remove symptoms, or update the doctors directory.

6.3 Lambda and Helper Functions

- resource = lambda f: os.path.join(BASE_DIR, f) — shortcut that turns any filename into its full path.
- centered_item(text) — creates a table cell item with center alignment, reused across all tables.
- confirm(parent, title, msg) — shows a Yes/No dialog and returns True if the user clicks Yes.

7. Testing and Results

7.1 Test Cases

#	Test Case	Input	Expected	Result
1	Valid patient login	Correct credentials	SymptomsWindow opens	PASS
2	Invalid login	Wrong password	Error message shown	PASS
3	Register new patient	New username	Account created in DB	PASS
4	Duplicate username	Existing username	Error: already exists	PASS
5	Short password	Less than 6 chars	Warning message	PASS
6	Predict with no symptoms	0 checkboxes	Warning: select at least 1	PASS
7	Flu/COVID diagnosis	Fever+Cough+Fatigue	flu_covid at 90%	PASS
8	Gastro diagnosis	Nausea+Vomiting+Diarrhea	gastro at 82%	PASS
9	Open Treatment window	Click Treatment button	TreatmentWindow opens	PASS
10	Save patient record	Click Save Record	Record stored with timestamp	PASS
11	Doctor login	Valid doctor credentials	DoctorDashboard opens	PASS
12	Search records	Type patient name	Table filters results	PASS

7.2 All Tests Passed

12 / 12 test cases passed. All edge cases handled with clear user feedback and no application crashes.

8. Conclusion and Future Work

8.1 Conclusion

Smart Symptom Checker successfully achieves all project objectives. It delivers a secure, offline, easy-to-use desktop application for symptom-based disease diagnosis. The dual-login system, rule-based AI engine, treatment recommendations, and doctor administration panel together make it a complete and practical healthcare tool.

The project demonstrates important software concepts including object-oriented design, event-driven programming, secure password storage, database management, and modular GUI development using PyQt5.

8.2 Advantages

- Fully offline — no internet connection needed at any time.
- Dual portal — separate and secure logins for patients and doctors.
- Instant diagnosis — results appear in milliseconds with a confidence percentage.
- Complete advice — every result includes medications, dosages, and home care tips.
- Doctor warnings — each condition tells the user when to seek emergency care.
- Extensible — doctors can add new symptoms without touching any code.
- Patient history — all records saved with timestamps and searchable by name.

8.3 Limitations

- Rule-based logic only — does not use machine learning, so accuracy depends on rule coverage.
- Covers 6 diseases only — unusual or complex conditions will return unknown.
- Windows desktop only — not available on mobile or web browsers.

8.4 Future Enhancements

- Add a machine learning model trained on medical data for improved accuracy.
- Expand to 20+ disease categories including chronic and skin conditions.
- Develop a mobile or web version for wider accessibility.
- Add vital sign inputs (temperature, blood pressure) to refine diagnosis.
- Generate printable prescription PDFs from the Doctor Dashboard.